

Analyse Factorielle des Correspondances (AFC) : Théorie et Implémentation

DUT : GI-SIR-IDIA / ESTBM USMS

Pr. Abdelaziz QAFFOU

Résumé

Ce document présente l'Analyse Factorielle des Correspondances (AFC), une méthode d'analyse factorielle adaptée aux tableaux de contingence. Nous expliquerons la théorie sous-jacente, fournirons des exemples d'implémentation en R et Python, et illustrerons la méthode avec un tableau de contingence concret.

Table des matières

1	Introduction à l'AFC	2
1.1	Domaine d'application	2
2	Théorie mathématique	2
2.1	Étapes principales	2
3	Tableau de contingence d'exemple	3
4	Implémentation en R	3
4.1	Code R complet pour l'AFC	3
4.2	Résultats de l'AFC en R	5
5	Implémentation en Python	5
5.1	Code Python complet pour l'AFC	5
6	Interprétation des résultats	9
6.1	Test du chi-deux	9
6.2	Analyse des axes factoriels	9
6.3	Interprétation des proximités	9
7	Extensions et variantes	9
7.1	AFC multiple (AFCM)	9
7.2	AFC sur tableau de fréquences	10
8	Comparaison R/Python	10
9	Conclusion	10

1 Introduction à l'AFC

L'Analyse Factorielle des Correspondances (AFC) est une méthode statistique développée par Jean-Paul Benzécri dans les années 1960 pour analyser les tableaux de contingence. Elle permet d'étudier les associations entre deux variables qualitatives en représentant les lignes et les colonnes dans un même espace factoriel.

1.1 Domaine d'application

- Analyse de tableaux de contingence (variables qualitatives)
- Étude des associations entre modalités de deux variables
- Visualisation des correspondances entre lignes et colonnes
- Analyse des données d'enquêtes et questionnaires

2 Théorie mathématique

Soit $N = (n_{ij})$ un tableau de contingence de dimensions $I \times J$, où n_{ij} représente l'effectif observé pour la ligne i et la colonne j .

2.1 Étapes principales

1. **Calcul des profils :**
 - Profils lignes : $p_{i|j} = \frac{n_{ij}}{n_{i\cdot}}$
 - Profils colonnes : $p_{j|i} = \frac{n_{ij}}{n_{\cdot j}}$
 - Profils marginaux : $p_i = \frac{n_{i\cdot}}{n}$ et $p_j = \frac{n_{\cdot j}}{n}$
2. **Matrice des écarts à l'indépendance :**

$$X_{ij} = \frac{n_{ij} - \frac{n_{i\cdot}n_{\cdot j}}{n}}{\sqrt{n_{i\cdot}n_{\cdot j}}}$$

3. **Décomposition en valeurs singulières (SVD) :**

$$X = U\Lambda V^T$$

où Λ est la matrice diagonale des valeurs singulières, U et V sont les matrices des vecteurs singuliers.

4. **Coordonnées factorielles :**
 - Coordonnées des lignes : $F = D_r^{-1/2}U\Lambda$
 - Coordonnées des colonnes : $G = D_c^{-1/2}V\Lambda$où D_r et D_c sont les matrices diagonales des profils marginaux lignes et colonnes.
5. **Inertie et pourcentage d'inertie :**

$$\text{Inertie totale} = \sum_{k=1}^K \lambda_k^2$$

$$\text{Inertie expliquée par l'axe } k = \frac{\lambda_k^2}{\sum \lambda_k^2} \times 100\%$$

3 Tableau de contingence d'exemple

Considérons un tableau de contingence fictif représentant la relation entre le niveau d'études et le type de lecture préféré pour 200 personnes interrogées.

TABLE 1 – Tableau de contingence : Niveau d'études $\otimes$ Type de lecture préféré

Niveau d'études	Type de lecture préféré				
	Journal	Roman	BD	Scientifique	Magazine
Bac	15	25	20	5	10
Licence	20	30	15	10	15
Master	10	20	10	25	10
Doctorat	5	10	5	30	5

TABLE 2 – Effectifs marginaux du tableau de contingence

	Effectifs lignes	Pourcentage
Bac	75	37.5%
Licence	90	45.0%
Master	75	37.5%
Doctorat	55	27.5%
	Effectifs colonnes	Pourcentage
Journal	50	25.0%
Roman	85	42.5%
BD	50	25.0%
Scientifique	70	35.0%
Magazine	40	20.0%

4 Implémentation en R

4.1 Code R complet pour l'AFC

```
1 # Chargement des packages nécessaires
2 library(FactoMineR)
3 library(factoextra)
4 library(ggplot2)
5 library(gplots)
6
7 # Création du tableau de contingence
8 contigence <- matrix(c(
9   15, 25, 20, 5, 10, # Bac
10  20, 30, 15, 10, 15, # Licence
11  10, 20, 10, 25, 10, # Master
12  5, 10, 5, 30, 5, # Doctorat
13 ), nrow = 4, byrow = TRUE)
14
15 rownames(contigence) <- c("Bac", "Licence", "Master", "Doctorat")
```

```

16  colnames(contingence) <- c("Journal", "Roman", "BD", "Scientifique",
17  "Magazine")
18
19  # Affichage du tableau de contingence
20  print("Tableau de contingence :")
21  print(contingence)
22
23  # Test du chi-deux d'indépendance
24  chi2_test <- chisq.test(contingence)
25  print("Test du chi-deux d'indépendance :")
26  print(chi2_test)
27
28  # AFC avec FactoMineR
29  afc_result <- CA(contingence, graph = FALSE)
30
31  # Résumé des résultats
32  print("Résumé de l'AFC :")
33  print(summary(afc_result))
34
35  # Valeurs propres et pourcentage d'inertie
36  valeurs_propres <- afc_result$eig
37  print("Valeurs propres et pourcentage d'inertie :")
38  print(valeurs_propres)
39
40  # Graphique des valeurs propres (Scree plot)
41  fviz_screplot(afc_result, addlabels = TRUE,
42  main = "Graphique des valeurs propres (Scree plot)")
43
44  # Biplot (lignes et colonnes)
45  fviz_ca_biplot(afc_result,
46  repel = TRUE,
47  title = "Biplot AFC - Lignes et colonnes")
48
49  # Graphique des lignes (niveaux d'études)
50  fviz_ca_row(afc_result,
51  col.row = "contrib",
52  gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
53  repel = TRUE,
54  title = "Projection des lignes (Niveau d'études)")
55
56  # Graphique des colonnes (types de lecture)
57  fviz_ca_col(afc_result,
58  col.col = "contrib",
59  gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
60  repel = TRUE,
61  title = "Projection des colonnes (Type de lecture)")
62
63  # Contribution des lignes et colonnes aux axes
64  print("Contribution des lignes aux axes :")
65  print(afc_result$row$contrib)
66
67  print("Contribution des colonnes aux axes :")
68  print(afc_result$col$contrib)
69
70  # Cos2 (qualité de représentation)
71  print("Cos2 des lignes (qualité de représentation) :")
72  print(afc_result$row$cos2)

```

```

73 print("Cos2 des colonnes (qualité de représentation) :")
74 print(afc_result$col$cos2)
75
76 # Tableau des distances du chi-deux
77 print("Distances du chi-deux entre lignes :")
78 dist_lignes <- dist(afc_result$row$coord)
79 print(as.matrix(dist_lignes))
80
81 # Visualisation du tableau de contingence avec heatmap
82 heatmap.2(contingence,
83 main = "Heatmap du tableau de contingence",
84 xlab = "Type de lecture",
85 ylab = "Niveau d'études",
86 trace = "none",
87 col = colorRampPalette(c("white", "blue", "darkblue"))(100),
88 dendrogram = "none",
89 Rowv = FALSE,
90 Colv = FALSE,
91 key = TRUE,
92 keysize = 1.5,
93 density.info = "none")
94
95 # Calcul des profils lignes et colonnes
96 profils_lignes <- prop.table(contingence, margin = 1)
97 print("Profils lignes (proportions par ligne) :")
98 print(round(profils_lignes, 3))
99
100 profils_colonnes <- prop.table(contingence, margin = 2)
101 print("Profils colonnes (proportions par colonne) :")
102 print(round(profils_colonnes, 3))
103
104 # Tableau des écarts à l'indépendance
105 tableau_independance <- chisq.test(contingence)$expected
106 ecarts <- contingence - tableau_independance
107 print("Écarts à l'indépendance (observés - attendus) :")
108 print(round(ecarts, 2))
109

```

Listing 1 – AFC en R

4.2 Résultats de l'AFC en R

Les principaux résultats de l'AFC en R incluent :

- Les valeurs propres et le pourcentage d'inertie expliquée
- Le test du chi-deux d'indépendance
- Les coordonnées factorielles des lignes et des colonnes
- Les contributions et cos2 (qualité de représentation)
- Le biplot qui superpose lignes et colonnes

5 Implémentation en Python

5.1 Code Python complet pour l'AFC

```

1 import numpy as np

```

```

2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.decomposition import TruncatedSVD
6 from scipy.stats import chi2_contingency
7 import prince # Pour l'AFC, installer avec: pip install prince
8
9 # Création du tableau de contingence
10 data = {
11     'Journal': [15, 20, 10, 5],
12     'Roman': [25, 30, 20, 10],
13     'BD': [20, 15, 10, 5],
14     'Scientifique': [5, 10, 25, 30],
15     'Magazine': [10, 15, 10, 5]
16 }
17
18 contingency = pd.DataFrame(data,
19 index=['Bac', 'Licence', 'Master', 'Doctorat'])
20
21 print("Tableau de contingence :")
22 print(contingency)
23 print()
24
25 # Test du chi-deux d'indépendance
26 chi2, p, dof, expected = chi2_contingency(contingency)
27 print(f"Test du chi-deux d'indépendance :")
28 print(f" Chi2 = {chi2:.4f}")
29 print(f" p-value = {p:.6f}")
30 print(f" Degrés de liberté = {dof}")
31 print()
32
33 # Calcul des profils
34 profils_lignes = contingency.div(contingency.sum(axis=1), axis=0)
35 print("Profils lignes (proportions par ligne) :")
36 print(profils_lignes.round(3))
37 print()
38
39 profils_colonnes = contingency.div(contingency.sum(axis=0), axis=1)
40 print("Profils colonnes (proportions par colonne) :")
41 print(profils_colonnes.round(3))
42 print()
43
44 # AFC avec le package prince
45 ca = prince.CA(n_components=2, random_state=42)
46 ca = ca.fit(contingency)
47
48 # Valeurs propres et inertie
49 print("Valeurs propres (inerties) :")
50 print(ca.eigenvalues_)
51 print()
52
53 print("Pourcentage d'inertie expliquée :")
54 print(ca.percentage_of_inertia_)
55 print()
56
57 print("Inertie expliquée cumulative :")
58 print(ca.cumulative_percentage_of_inertia_)
59 print()

```

```

60
61 # Coordonnées des lignes et colonnes
62 print("Coordonnées des lignes (niveaux d'études) :")
63 print(ca.row_coordinates(contingence).round(4))
64 print()
65
66 print("Coordonnées des colonnes (types de lecture) :")
67 print(ca.column_coordinates(contingence).round(4))
68 print()
69
70 # Contributions
71 print("Contributions des lignes aux axes :")
72 contrib_lignes = ca.row_contributions_.round(4)
73 print(contrib_lignes)
74 print()
75
76 print("Contributions des colonnes aux axes :")
77 contrib_colonnes = ca.column_contributions_.round(4)
78 print(contrib_colonnes)
79 print()
80
81 # Graphique des valeurs propres (Scree plot)
82 plt.figure(figsize=(10, 6))
83 inerties = ca.eigenvalues_
84 plt.bar(range(1, len(inerties) + 1), inerties, alpha=0.7, align='
center')
85 plt.xlabel('Axe factoriel')
86 plt.ylabel('Valeur propre (Inertie)')
87 plt.title('Scree plot - Valeurs propres')
88 plt.grid(True, alpha=0.3)
89 plt.show()
90
91 # Biplot AFC
92 fig, ax = plt.subplots(figsize=(12, 10))
93
94 # Projection des lignes
95 row_coords = ca.row_coordinates(contingence)
96 ax.scatter(row_coords[0], row_coords[1], c='blue', s=200, alpha=0.7,
label='Lignes')
97 for idx, row in row_coords.iterrows():
98 ax.annotate(idx, (row[0], row[1]), fontsize=12, color='blue',
99 xytext=(5, 5), textcoords='offset points')
100
101 # Projection des colonnes
102 col_coords = ca.column_coordinates(contingence)
103 ax.scatter(col_coords[0], col_coords[1], c='red', s=200, alpha=0.7,
label='Colonnes')
104 for idx, row in col_coords.iterrows():
105 ax.annotate(idx, (row[0], row[1]), fontsize=12, color='red',
106 xytext=(5, 5), textcoords='offset points')
107
108 # Lignes de référence
109 ax.axhline(y=0, color='k', linestyle='--', alpha=0.3)
110 ax.axvline(x=0, color='k', linestyle='--', alpha=0.3)
111
112 # Configuration du graphique
113 ax.set_xlabel(f'Axe 1 ({ca.percentage_of_inertia_[0]:.1f}%)',
fontsize=12)

```

```

114     ax.set_ylabel(f'Axe 2 ({ca.percentage_of_inertia_[1]:.1f}%)',
115     fontsize=12)
116     ax.set_title('Biplot AFC - Niveau d\'études & Type de lecture',
117     fontsize=14)
118     ax.legend(fontsize=12)
119     ax.grid(True, alpha=0.3)
120     plt.tight_layout()
121     plt.show()
122
123     # Graphique séparé des lignes
124     plt.figure(figsize=(10, 8))
125     plt.scatter(row_coords[0], row_coords[1], c='blue', s=300)
126     for idx, row in row_coords.iterrows():
127         plt.annotate(idx, (row[0], row[1]), fontsize=14, color='blue',
128         xytext=(5, 5), textcoords='offset points')
129
130     plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
131     plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
132     plt.xlabel(f'Axe 1 ({ca.percentage_of_inertia_[0]:.1f}%)', fontsize
133     =12)
134     plt.ylabel(f'Axe 2 ({ca.percentage_of_inertia_[1]:.1f}%)', fontsize
135     =12)
136     plt.title('Projection des lignes (Niveau d\'études)', fontsize=14)
137     plt.grid(True, alpha=0.3)
138     plt.tight_layout()
139     plt.show()
140
141     # Graphique séparé des colonnes
142     plt.figure(figsize=(10, 8))
143     plt.scatter(col_coords[0], col_coords[1], c='red', s=300)
144     for idx, row in col_coords.iterrows():
145         plt.annotate(idx, (row[0], row[1]), fontsize=14, color='red',
146         xytext=(5, 5), textcoords='offset points')
147
148     plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
149     plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
150     plt.xlabel(f'Axe 1 ({ca.percentage_of_inertia_[0]:.1f}%)', fontsize
151     =12)
152     plt.ylabel(f'Axe 2 ({ca.percentage_of_inertia_[1]:.1f}%)', fontsize
153     =12)
154     plt.title('Projection des colonnes (Type de lecture)', fontsize=14)
155     plt.grid(True, alpha=0.3)
156     plt.tight_layout()
157     plt.show()
158
159     # Heatmap du tableau de contingence
160     plt.figure(figsize=(10, 8))
161     sns.heatmap(contingence, annot=True, fmt='d', cmap='Blues',
162     cbar_kws={'label': 'Effectif'})
163     plt.title('Heatmap du tableau de contingence', fontsize=14)
164     plt.xlabel('Type de lecture', fontsize=12)
165     plt.ylabel('Niveau d\'études', fontsize=12)
166     plt.tight_layout()
167     plt.show()
168
169     # Heatmap des écarts à l'indépendance
170     ecarts = contingence - expected
171     plt.figure(figsize=(10, 8))

```



```

166     sns.heatmap(ecarts, annot=True, fmt='.1f', cmap='RdBu_r', center=0,
167     cbar_kws={'label': 'Écart à l\'indépendance'})
168     plt.title('Écarts à l\'indépendance (observé - attendu)', fontsize
=14)
169     plt.xlabel('Type de lecture', fontsize=12)
170     plt.ylabel('Niveau d\'études', fontsize=12)
171     plt.tight_layout()
172     plt.show()
173
174     # Analyse complémentaire : tableau des contributions au chi-deux
175     chi2_table = (contingence - expected)**2 / expected
176     print("Contributions au chi-deux (par cellule) :")
177     print(chi2_table.round(4))
178     print()
179
180     print(f"Somme des contributions = {chi2_table.sum().sum():.4f}")
181     print(f"Chi-deux calculé = {chi2:.4f}")
182

```

Listing 2 – AFC en Python

6 Interprétation des résultats

Pour notre exemple de tableau de contingence, on peut interpréter les résultats comme suit :

6.1 Test du chi-deux

Le test du chi-deux permet de vérifier s'il existe une association significative entre les variables. Dans notre cas, avec une p-value < 0.05 , on rejette l'hypothèse d'indépendance.

6.2 Analyse des axes factoriels

- **Axe 1** : Oppose généralement les lectures "scientifiques" (associées aux niveaux d'études élevés) aux lectures "grand public" (journaux, magazines, BD).
- **Axe 2** : Peut opposer les lectures de divertissement (romans, BD) aux lectures d'information (journaux, magazines).

6.3 Interprétation des proximités

- La proximité entre "Doctorat" et "Scientifique" indique une forte association.
- La proximité entre "Bac" et "BD/Roman" suggère une préférence pour ces types de lecture.
- La position centrale de "Licence" et "Master" indique un profil plus équilibré.

7 Extensions et variantes

7.1 AFC multiple (AFCM)

L'AFCM étend l'AFC à plus de deux variables qualitatives. Elle analyse un tableau de Burt ou un tableau disjonctif complet.

7.2 AFC sur tableau de fréquences

L'AFC peut également s'appliquer à des tableaux de fréquences relatives (pourcentages) plutôt qu'à des effectifs absolus.

8 Comparaison R/Python

TABLE 3 – Comparaison des approches R et Python pour l'AFC

Aspect	R (FactoMineR)	Python (prince)
Installation	<code>install.packages("FactoMineR")</code>	<code>pip install prince</code>
Fonction principale	<code>CA()</code>	<code>prince.CA()</code>
Visualisation	Excellente avec <code>factoextra</code>	Manuel avec <code>matplotlib</code>
Sorties détaillées	Très complètes (contributions, cos2, etc.)	Complètes mais format différent
Intégration tidyverse	Bonne avec <code>dplyr</code>	Bonne avec <code>pandas</code>
Communauté	Statistique francophone	Machine Learning

9 Conclusion

L'AFC est une méthode puissante pour analyser les associations entre variables qualitatives à travers un tableau de contingence. Elle permet de :

- Visualiser simultanément les modalités de deux variables
- Identifier les associations significatives
- Interpréter la structure sous-jacente des données
- Compléter l'analyse du chi-deux par une représentation géométrique

Les implémentations en R et Python offrent des fonctionnalités similaires, avec R offrant généralement plus d'options de visualisation prêtes à l'emploi, et Python s'intégrant mieux dans des pipelines de data science.

Références

- Benzécri, J.-P. (1973). L'analyse des données. Tome 1 : La taxinomie. Tome 2 : L'analyse des correspondances.
- Lebart, L., Morineau, A., & Warwick, K. M. (1984). Multivariate descriptive statistical analysis.
- Husson, F., Lê, S., & Pagès, J. (2017). Exploratory multivariate analysis by example using R.
- Greenacre, M. (2007). Correspondence analysis in practice.
- Documentation FactoMineR : <https://cran.r-project.org/web/packages/FactoMineR/>
- Documentation prince : <https://github.com/MaxHalford/prince>